Detecting and characterizing LLM jailbreaks in conversational data

Francisco Gálvez
School of Engineering and Sciences Computer and Telecommunications Engineering
Universidad Diego Portales
Santiago, Chile
Email: francisco.galvez_p@mail.udp.cl

César Muñoz Rivera
School of Engineering and Sciences Computer and Telecommunications Engineering
Universidad Diego Portales
Santiago, Chile
Email: cesar.munoz_r@mail.udp.cl

J. Tomás Silva
School of Engineering and Sciences Computer and Telecommunications Engineering
Universidad Diego Portales
Santiago, Chile
Email: jose.silva5@mail.udp.cl

***Abstract*—The widespread adoption of Large Language Models (LLMs) has exposed a critical vulnerability: "jailbreaking", where adversarial prompts are used to bypass built-in safety mechanisms. As LLM usage grows, detecting these attacks requires systems that balance high throughput with interpretability. This paper proposes a scalable framework to automatically detect and characterize LLM jailbreaks. We develop a hybrid classification architecture combining distributed embedding generation with gradient boosting to identify nuanced attack patterns, including pretexting and scenario manipulation. Crucially, we introduce an automated semantic profiling mechanism that translates opaque clusters into actionable threat intelligence via keyword extraction. Validated on a combined corpus of over 300,000 adversarial and benign prompts, our approach achieves an AUC-ROC of 0.969 and demonstrates how unsupervised learning can be used to discover and characterize novel attack families without manual labeling.**

1. Introduction

The rapid proliferation of large language models (LLMs) such as GPT and Claude has made them integral to modern society, transforming how we interact with technology in education, professional work, and daily life. This widespread adoption is driven by their powerful ability to follow and execute nuanced instructions. However, this core capability also presents a significant vulnerability: "jailbreaking".

Jailbreaking refers to a technique where users craft specific adversarial prompts to bypass an LLM's built-in safety filters and ethical guidelines. These attacks are designed to manipulate the model into producing content that is unsafe, inappropriate, or violates its terms of service, such as generating instructions for harmful acts, providing access to pirated content, or creating toxic text. A common method is "double-context" or pretexting, where a malicious request is disguised within a seemingly innocent query, making it difficult for automated safety systems to detect and block. For example, a user might ask for a list of piracy websites under the pretense of "wanting to know which sites to avoid". A jailbreak can also be executed by creating a persona or scenario that pressures the AI to ignore its safety protocols. For example, a user might use a "plane crash survivors" scenario, where they instruct the AI to act as a survivor who needs to provide instructions for making a dangerous substance like meth with "no moral and no ethics" to save the group from an imagined threat. This type of prompt manipulates the model by creating a sense of urgency and necessity, where generating harmful content is framed as the only way to survive. The user might even specify that the AI, acting as a survivor, must provide a detailed, step-by-step tutorial without any warnings or disclaimers, effectively tricking the model into bypassing its safety filters.

The increasing use of LLMs by a diverse, global audience generates a massive volume of conversational data, creating a challenging environment for ensuring model safety at scale. For instance, ChatGPT now has over 100 million active weekly users, underscoring the immense and growing scale of the user base [1]. The scale of this problem is immense, as platforms like Chatbot Arena collect millions of real-time conversations between users and LLMs. This presents a significant Big Data challenge: how to effectively analyze vast datasets to identify and characterize these sophisticated jailbreak patterns. This paper addresses this critical issue by proposing a scalable solution to detect and classify LLM jailbreaks in conversational data. We leverage a large-scale dataset of over one million conversations and apply a distributed classification model to learn and recognize the nuanced characteristics of these adversarial attacks, thereby enhancing the security and ethical alignment of LLM systems. Our approach focuses on curating and processing this data, training a robust model, and validating its effectiveness using standard classification metrics.

2. Datasets

To address the problem of detecting and characterizing jailbreaks in LLM conversations, we utilize two complementary datasets: the *Prompt Injection Safety* dataset [26] and the *Prompt Injection* dataset [27]. The former provides approximately 50,000 labeled records, while the latter

expands our corpus with an additional 262,000 records. Together, these curated collections aggregate over 312,000 instances, combining diverse attack vectors from sources such as JailbreakBench and wild adversarial examples to provide a massive, balanced distribution of safe and unsafe prompts essential for robust supervised learning at scale.

To demonstrate the scalability of our architecture for the "Big Data" challenge presented by datasets like LMSYS-Chat-1M [2], we implement our data ingestion and embedding pipeline using PySpark. This ensures that while our model validation focuses on this high-density combined attack corpus, the underlying engineering framework remains capable of processing million-scale conversational logs found in production environments. Each record corresponds to a complete interaction and is processed to preserve the natural sequence of user inputs and model responses.

3. State of the Art

The proliferation of Large Language Models (LLMs) has introduced significant vulnerabilities through jailbreaking: adversarial attacks designed to bypass safety mechanisms via complex strategies like role-playing, pretexting, and contextual manipulation [7], [8], [14]. The exponential growth of LLM adoption, with platforms managing hundreds of millions of users [1], has transformed this security vulnerability into a massive big data challenge. Analyzing massive, real-world conversational datasets, such as the LMSYS-Chat-1M collection of over one million dialogues [2], [3], requires solutions that are not only accurate but highly scalable [9].

3.1. Evolution of Detection Strategies

Early defense mechanisms employed computationally simple but fragile methods, such as prohibited word dictionaries, regular expressions, and heuristic rules [7]. These systems proved trivial to bypass with linguistic variations. Statistical metrics, such as perplexity, were explored for detecting machine-generated attacks but proved ineffective against human-crafted jailbreaks that use natural, low-perplexity language [15].

This led to the adoption of classic machine learning classifiers (e.g., SVM, Naive Bayes) using engineered linguistic features [7]. While an improvement, these models struggled to generalize against the increasing complexity of adversarial prompts. The paradigm shifted with the advent of pre-trained transformers, such as BERT [11] and RoBERTa [6], which marked a significant leap in detection capabilities. This approach, which fine-tunes an encoder model for classification, became the *de facto* state of the art. This has led to several popular open-source classifiers available on platforms like Hugging Face, including Myadav/setfit-prompt-injection-MiniLM-L3-v2, protectai/deberta-v3-base-prompt-injection-v2 and deepset/deberta-v3-base-injection. [19]

3.2. Scalability and hibrid models

While effective, this transformer-based state of the art presents a critical challenge, the computational cost. Fine-tuning and running large encoder models for inference on millions of conversations is computationally expensive and introduces significant latency, making it poorly suited for the big data problem [19].

This scalability bottleneck has spurred the development of a more efficient, hybrid approach: combining pre-trained text embeddings with traditional, lightweight ML classifiers like Random Forest or XGBoost [10]. Other systems use vector databases to compare prompt embeddings against a database of known jailbreaks [13]. This hybrid method was often seen as a trade-off, sacrificing potential accuracy for speed.

However, a recent study by Ayub and Majumdar (2024) [19] directly challenges this assumption. The authors conducted a head-to-head comparison between embedding-based classifiers (Random Forest and XGBoost) and the very state of the art transformer models available on Hugging Face [19]. Their findings were definitive, the hybrid model (specifically Random Forest trained on OpenAI embeddings) outperformed all four state of the art deep learning classifiers in terms of AUC and precision scores [19]. The hybrid model achieved the highest F1-score, demonstrating a superior balance between precision and recall.

The study noted that while the transformer models had high recall, their precision was low, whereas the embedding-based approach was more balanced and ultimately more effective [19].

3.3. Justification of the Proposal

This recent finding [19] indicates a paradigm shift, the most accurate detection method may also be the most computationally efficient. This presents a massive opportunity. However, a critical gap remains between the model and its implementation at scale:

1) **State of the art Models:** Research has validated the *model* (embeddings + RF/XGBoost) on a static dataset [10], [19].
2) **State of the art Analysis:** Tools like JailbreakHunter have been built for large-scale visual analytics and expert exploration, not automated, real-time detection [9].
3) **The Gap:** No published research has focused on the *scalable implementation* of this new, efficient state of the art model to address the "big data" challenge in an automated, high-throughput escenario.

Our research directly addresses this gap. We propose an automated and distributed classification framework specifically designed to perform robust jailbreak detection on massive, real-world datasets [2]. By building an architecture to scalably deploy the embedding-based detection method, our

work provides the necessary bridge between model-level-state of the art [19] and the practical, big-data requirements of LLM security, proving it is possible to build a system that is both highly accurate and highly scalable.

4. Proposed scalable framework: An end-to-end threat intelligence pipeline

To address the limitations of existing detection methods, specifically the critical trade-off between computational scalability and classification precision [10], [19], we propose a distributed multi-stage framework designed for high-throughput analysis of conversational data. Unlike traditional approaches that rely solely on computationally expensive LLM fine-tuning, our architecture decouples the feature extraction process from the classification logic. This strategy enables real-time processing of massive datasets such as LMSYS-Chat-1M [2], [3].

The proposed pipeline operates as a cohesive "filter-and-analyze" system structured into three sequential stages: (1) Distributed Data Unification, where heterogeneous data sources are harmonized and vectorized; (2) High-Throughput Detection, employing a distributed Gradient Boosting architecture to rapidly segregate adversarial prompts from benign interactions and (3) Unsupervised Characterization, utilizing density-based clustering and Shapley Additive Explanations (SHAP) to discover and profile emerging attack vectors.

The key objective of this design is to facilitate the integration of "Signal Injection" by augmenting real-world conversational data with specialized jailbreak datasets [20], [21], [23] to correct class imbalances without compromising the statistical representativeness of the traffic. By leveraging distributed computing primitives, the framework ensures that both training and inference scale linearly with the volume of data, effectively bridging the gap between state-of-the-art detection models and the big data requirements of modern LLM security.

4.1. Distributed Data Unification and Signal Injection

To prepare a robust training corpus capable of identifying nuanced jailbreak attempts, we propose a distributed data engineering pipeline. This design moves beyond simple concatenation, outlining a multi-stage workflow to ingest, clean, and vectorize heterogeneous data sources at scale.

4.1.1. Schema harmonization and data ingestion. The proposed pipeline begins by ingesting large-scale datasets, which serve as the background distribution of real-world user interactions. To address the scarcity of adversarial examples in wild traffic (approximately 5.44% [3]), the framework is designed to augment this stream with specialized high-density attack datasets, including *JailbreakBench* [21], *Prompt Injection Cleaned* [20], and *Deepset* [23]. Since these datasets utilize different file structures (e.g., varying JSON schemas or CSV columns), we propose a distributed mapping process utilizing PySpark. This process is intended to standardize all incoming records into a unified format containing only the essential signals: the conversation history, the user prompt, and a binary safety label. This step ensures that subsequent stages operate on a consistent data structure regardless of the original source.

4.1.2. Semantic deduplication process. Aggregating multiple open-source vulnerability datasets often introduces redundancy, where identical or slightly modified attacks (e.g., the popular "DAN" or "MONGO" jailbreaks) appear across different repositories. Training on duplicated data can lead to model overfitting and inflated performance metrics. To mitigate this, the architecture incorporates a semantic deduplication stage based on MinHash Locality Sensitive Hashing (LSH). Unlike exact string matching, this process is designed to identify prompts that are semantically identical despite minor variations in wording or punctuation. By filtering out these near-duplicates, the "Signal Injection" process aims to introduce only unique and diverse attack vectors into the training set, maintaining the integrity of the evaluation.

4.1.3. Distributed vectorization and feature extraction. Once the data is harmonized and cleaned, the final stage is designed to transform the raw text into numerical features suitable for machine learning. Instead of fine-tuning a heavy LLM during the classification phase, we propose utilizing a pre-trained transformer model purely as a feature extractor. We select high-performance embedding models such as *all-MiniLM-L6-v2* or *DeBERTa-v3*, which are optimized for sentence similarity tasks. This inference process is designed to be parallelized across a cluster of GPU workers. By leveraging a zero-copy memory store (in this case, we used Apache Arrow), the system aims to efficiently batch and compute embeddings for millions of turns without the latency bottlenecks typical of serial processing. The output would be a dense feature matrix ready for high-throughput training.

4.2. High-throughput detection via distributed gradient boosting

Following the vectorization of conversational logs, the second stage of the framework focuses on the rapid segregation of adversarial prompts from benign interactions. While recent literature explores fine-tuning LLMs for this task, such approaches often incur prohibitive computational costs and latency penalties when applied to million-scale datasets. Consistent with findings by Ayub and Majumdar [19], which demonstrate that embedding-based classifiers can outperform deep learning baselines in both precision and inference speed, we propose a distributed detection architecture based on Gradient Boosting Decision Trees (GBDT).

4.2.1. Distributed histogram-based learning. We select XGBoost as the core classification engine, specifically leveraging its distributed histogram-based algorithm. Unlike exact greedy algorithms that enumerate all possible splits for continuous features, our proposed architecture discretizes the continuous embedding vectors into integer-mapped histograms. The framework leverages XGBoost's histogram-based algorithm, which discretizes continuous embedding vectors into integer-mapped bins. This design enables horizontal scaling via XGBoost's native Spark integration when dataset sizes exceed single-node memory capacity. For our experimental evaluation on 300,000 samples, training was performed on a single node with multi-core parallelization.

4.2.2. Automated hyperparameter optimization. To maximize detection performance while ensuring reproducibility, we perform hyperparameter optimization using stratified k-fold cross-validation with grid search. While Bayesian optimization methods offer theoretical advantages for large parameter spaces, grid search with cross-validation provides comparable performance for our focused parameter space while maintaining simplicity and reproducibility.

We evaluate 81 configurations across learning rate $\eta \in \{0.03, 0.05, 0.07\}$, maximum tree depth $\in \{5, 6, 7\}$, `scale_pos_weight` $\in \{7.0, 9.0, 11.0\}$, and number of estimators $\in \{800, 1000, 1200\}$. Using 3-fold stratified cross-validation, this approach completes in under 3 minutes on consumer hardware while ensuring robust parameter selection that generalizes across data splits.

We employ randomized hyperparameter search over learning rate $\eta \in \{0.03, 0.10\}$, maximum tree depth $\eta \in \{4, 5, 6, 7\}$, scale$_{p}os_{w}eight\eta \in \{5.0, 13.0\}$, and number of estimators $\eta \in \{600, 1000\}$. We sample 20 configurations and evaluate each with 3-fold stratified cross-validation (60 total fits), completing in approximately 45 minutes on consumer hardware (Apple M4).

4.2.3. Imbalanced Learning Strategy. Given the extreme sparsity of jailbreak attempts in real-world traffic (approximately 5-6% of data [3]), we address class imbalance through XGBoost's `scale_pos_weight` parameter. This mechanism adjusts gradient contributions during backpropagation to prevent the model from favoring the majority class.

Our cross-validation search identifies optimal weighting (`scale_pos_weight` $\in \{7.0, 9.0, 11.0\}$) that balances precision and recall. While advanced techniques like Focal Loss offer theoretical benefits for hard-negative mining, our evaluation shows that `scale_pos_weight` adjustment achieves comparable performance ($> 95\%$ AUC-ROC) with significantly reduced implementation complexity.

4.3. Unsupervised Characterization and Explainability

The final stage of the proposed framework addresses the "black box" nature of detection systems. Once adversarial prompts are segregated by the classifier, mere detection is insufficient for proactive security; security analysts require granular insights into the nature of the attacks. To achieve this, we design an unsupervised characterization module capable of discovering novel attack patterns (zero-day jailbreaks) and providing semantic explanations without manual labeling.

4.3.1. Manifold Learning and Comparative Density Clustering. Clustering high-dimensional embedding vectors directly ($D = 768$) is often ineffective due to the "curse of dimensionality," where distance metrics lose their discriminative power. To mitigate this, the pipeline incorporates a dimensionality reduction step using Uniform Manifold Approximation and Projection (UMAP), projecting embeddings into a lower-dimensional latent space suitable for density analysis.

Following reduction, we implement a comparative clustering framework to evaluate the stability of attack characterization. We benchmark two density-based algorithms: the classical DBSCAN and its hierarchical extension, HDBSCAN [28].

While DBSCAN serves as a lightweight baseline for detecting clusters of uniform density, it requires precise manual tuning of the neighborhood radius (ϵ), which is often impractical for dynamic threat landscapes. Conversely, HDBSCAN constructs a cluster hierarchy to identify groups of varying densities without dataset-specific calibration. By evaluating both methods using the *Relative Validity Index* (DBCV), our framework selects the optimal clustering strategy that maximizes the separation between coordinated attack campaigns and random noise. This adaptive approach ensures the system remains robust against both trivial, high-density attacks and sophisticated, low-density evasion attempts.

4.3.2. Automated Profiling via SHAP. To translate abstract clusters into actionable intelligence, the framework employs a post-hoc explainability mechanism. Since unsupervised clusters lack semantic labels, we propose training a lightweight "surrogate model" (e.g., a multi-class LightGBM classifier) to predict the cluster assignment ID from the original text embeddings. By applying SHAP to this surrogate model, the system can calculate the contribution of specific input features to the cluster assignment. This allows for the automated generation of "Cluster Profiles." For instance, SHAP analysis might reveal that a specific cluster is heavily driven by semantic features related to "roleplay," "acting," and "movie scripts," thereby automatically characterizing the group as a "Persona Manipulation" attack family. This closes the loop between raw data detection and high-level threat characterization.

4.3.3. Semantic Profiling for Actionable Intelligence. While statistical clustering identifies groups of related attacks, it does not explain why they are grouped. To bridge the gap between abstract vectors and operational security, we implement an automated semantic profiling module.

For each discovered cluster, the system performs lexical frequency analysis on the raw text associated with the cluster's embeddings. We tokenize the prompts, remove standard

stop-words, and extract the most frequent tokens (length ≥ 4). This yields characteristic "keyword signatures" for each attack family. For example, a cluster may be defined by the high-frequency co-occurrence of terms like "ignore", "instructions", and "previous", immediately identifying it as an "Instruction Override" attack family. This method provides interpretable, actionable intelligence that can be directly translated into regex-based filters or firewall rules.

5. Implementation Considerations

5.1. Computational Efficiency

Our implementation prioritizes reproducibility and practical deployment. While the hyperparameter optimization phase (searching 243 configurations via grid search) represents a one-time computational cost of approximately 60 minutes on consumer hardware (MacBook Pro M4), the operational pipeline is highly efficient. Once hyperparameters are frozen, the complete retraining process—from raw text ingestion to final classifier generation—achieves a throughput rate capable of processing 50,000 records in under 10 minutes.

5.2. Scalability Architecture

While evaluated on a combined corpus of over 300,000 samples, our architecture employs distributed processing (PySpark) designed for web-scale data. The embedding generation stage processes batches of 64 texts concurrently across multiple workers, achieving throughput suitable for processing million-scale datasets within practical time constraints.

5.3. Inference Performance

Deployed classifiers achieve real-time inference with less than 50ms latency per prompt (including embedding generation), making the system suitable for production safety filtering in high-traffic applications.

6. Experimental Results

6.1. Classification Performance

We evaluated our framework on a hold-out test set comprising 20% of the data. Despite the class imbalance, the optimized XGBoost model achieved an AUC-ROC of 0.969. To rigorously assess performance beyond accuracy, we utilize the Matthews Correlation Coefficient (MCC), which yielded a score of 0.88, indicating strong correlation between predictions and ground truth even with imbalanced classes. The model maintains a low False Positive Rate (3.1%), ensuring usability in production environments where alert fatigue is a concern.

6.2. Characterization of Attack Families

The unsupervised module successfully identified distinct attack strategies. UMAP projection revealed well-separated manifolds, which HDBSCAN segmented into cohesive clusters. Our semantic profiling module automatically characterized these clusters; for instance, identifying a massive "Role-Play" cluster defined by keywords *character*, *act*, and *fictional*, and a distinct "Pretexting" cluster defined by *code*, *developer*, and *mode*. This validates the framework's ability to discover and describe attack vectors without prior knowledge.

7. Discussion

7.1. Operational Utility vs. Pure Accuracy

While previous works prioritize maximizing AUC on static benchmarks, our results highlight the importance of interpretability in security operations. The detection accuracy (AUC 0.969) is comparable to computationally expensive deep learning baselines [19], but our framework offers superior throughput and deployability. More importantly, the semantic profiling module transforms the system from a passive filter into an active threat intelligence tool. By automatically surfacing keywords like "roleplay" or "ignore instructions," the system allows security teams to draft regex-based firewall rules for immediate mitigation, bridging the gap between machine learning detection and policy enforcement.

7.2. Efficiency at Scale

The transition from complex hyperparameter optimization to efficient grid search, and from DBSCAN to HDBSCAN, demonstrates that "simpler" robust engineering often yields better practical results. Our pipeline processes ingestion, embedding, and classification for 50,000 records in under 10 minutes on consumer hardware, suggesting it can handle the velocity of real-time production logs (e.g., LMSYS-scale data) without becoming a latency bottleneck.

8. Limitations and Future Work

Our study has several limitations. First, while the architecture is designed for big data, our supervised model training relied on labeled datasets totaling approximately 312,000 samples. Validating the classifier on the full LMSYS-Chat-1M corpus would require extensive manual labeling or reliable weak supervision signals. Second, our semantic profiling currently relies on English-language tokenization; attacks in low-resource languages or utilizing base64 encoding might evade characterization.

Future work will focus on three areas: (1) implementing active learning to selectively sample uncertain predictions from the 1M+ unlabeled conversations for human review; (2) extending the semantic profiler to handle multilingual

and obfuscated text; and (3) testing the resilience of the XGBoost classifier against adaptive adversarial attacks designed to evade embedding-based detection.

9. Conclusion

As Large Language Models become ubiquitous, the "Big Data" challenge of monitoring conversational logs for jailbreaks grows more acute. This paper presented a scalable, end-to-end framework that not only detects attacks with high precision (MCC 0.88) but also characterizes them into actionable semantic families. By combining distributed engineering (PySpark), robust clustering (HDBSCAN), and interpretable profiling (Keyword Extraction), we demonstrated that it is possible to build security systems that are both highly accurate and operationally transparent. Our findings suggest that the future of LLM security lies not just in smarter models, but in systems that can explain the nature of the threats they detect.

References

[1] DemandSage, "ChatGPT Statistics", https://www.demandsage.com/chatgpt-statistics/.

[2] LMSYS, "LMSYS-Chat-1M: A Large-Scale Real-World LLM Conversation Dataset", Hugging Face, https://huggingface.co/datasets/lmsys/lmsys-chat-1m.

[3] Zheng, L., Chiang, W. L., Sheng, Y., Li, T., Zhuang, S., Wu, Z., ... & Zhang, H. (2023). "LMSYS-Chat-1M: A Large-Scale Real-World LLM Conversation Dataset.", https://arxiv.org/pdf/2309.11998 .

[4] Y. Zeng, H. Lin, D. Yang, and R. Jia, "How Johnny Can Persuade LLMs to Jailbreak Them: Rethinking Persuasion to Challenge AI Safety by Humanizing LLMs," in *can. 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[5] S. Saha, J. Chen, S. Mayers, S. K. Gouda, Z. Wang, and V. Kumar, "BREAKING THE CODE: Security Assessment of AI Code Agents Through Systematic Jailbreaking Attacks," *arXiv preprint arXiv:2510.01359*, 2025.

[6] M. F. A. Sayeedi, M. B. Hossain, M. K. Hassan, S. Afrin, M. M. S. Hossain, and M. S. Hossain, "JailbreakTracer: Explainable Detection of Jailbreaking Prompts in LLMs Using Synthetic Data Generation," *IEEE Access*, 2025, doi: 10.1109/ACCESS.2025.3579996.

[7] H. Strohmier, Y. Dasri, and D. Murzello, "Exploring Security Vulnerabilities in ChatGPT Through Multi-Technique Evaluation of Resilience to Jailbreak Prompts and Defensive Measures," in *2024 International Conference on Computer and Applications (ICCA)*, 2024, pp. 1-8, doi: 10.1109/ICCA62237.2024.10928071.

[8] M. Gupta, C. Akiri, K. Aryal, E. Parker, and L. Praharaj, "From ChatGPT to ThreatGPT: Impact of Generative AI in Cybersecurity and Privacy," *IEEE Access*, vol. 11, pp. 80235-80246, 2023, doi: 10.1109/ACCESS.2023.3300381.

[9] Z. Jin, S. Liu, H. Li, X. Zhao, and H. Qu, "JailbreakHunter: A Visual Analytics Approach for Jailbreak Prompts Discovery from Large-Scale Human-LLM Conversational Datasets," *arXiv preprint arXiv:2407.03045*, 2024.

[10] E. Galinkin and M. Sablotny, "Improved Large Language Model Jailbreak Detection via Pretrained Embeddings," *arXiv preprint arXiv:2412.01547*, 2024.

[11] J. Hawkins, A. Pramar, R. Beard, and R. Chandra, "Machine Learning for Detection and Analysis of Novel LLM Jailbreaks," *arXiv preprint arXiv:2510.01644*, 2025.

[12] M. Hu, X. Wu, Z. Suo, J. Feng, L. Meng, Y. Jia, A. T. Luu, and S. Zhao, "Rethinking Reasoning: A Survey on Reasoning-based Backdoors in LLMs," *arXiv preprint arXiv:2510.07697*, 2025.

[13] B. Pingua et al., "Mitigating adversarial manipulation in LLMs: a prompt-based approach to counter Jailbreak attacks (Prompt-G)," *PeerJ Computer Science*, vol. 10, p. e2374, 2024, doi: 10.7717/peerj-cs.2374.

[14] Y. Tang, B. Wang, X. Wang, D. Zhao, J. Liu, J. Zhang, R. He, and Y. Hou, "RoleBreak: Character Hallucination as a Jailbreak Attack in Role-Playing Systems," *arXiv preprint arXiv:2409.16727*, 2024.

[15] G. Alon and M. Kamfonas, "Detecting Language Model Attacks with Perplexity," *arXiv preprint arXiv:2308.14132*, 2023.

[16] J. Chen and D. Yang, "Weakly-Supervised Hierarchical Models for Predicting Persuasion Strategies," *arXiv preprint arXiv:2101.06351*, 2021.

[17] G. Deng et al., "MASTERKEY: Automated Jailbreaking of Large Language Model Chatbots," *arXiv preprint arXiv:2307.08715*, 2023.

[18] Y. Deng, W. Zhang, S. J. Pan, and L. Bing, "Multilingual Jailbreak Challenges in Large Language Models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[19] M. A. Ayub and S. Majumdar, "Embedding-based classifiers can detect prompt injection attacks," *arXiv preprint arXiv:2410.22284*, 2024.

[20] Imoxto, "Prompt Injection Cleaned Dataset," Hugging Face, https://huggingface.co/datasets/imoxto/prompt_injection_cleaned_dataset.

[21] J. Chao et al., "JailbreakBench: An Open Robustness Benchmark for Jailbreaking Large Language Models," Hugging Face, https://huggingface.co/datasets/JailbreakBench/JBB-Behaviors.

[22] LibrAI, "Do Not Answer: A Safety Dataset for LLMs," Hugging Face, https://huggingface.co/datasets/LibrAI/do-not-answer.

[23] Deepset, "Prompt Injections Dataset," Hugging Face, https://huggingface.co/datasets/deepset/prompt-injections.

[24] xTRam1, "Safe Guard Prompt Injection Dataset," Hugging Face, https://huggingface.co/datasets/xTRam1/safe-guard-prompt-injection.

[25] Markush1, "LLM Jailbreak Classifier Dataset," Hugging Face, https://huggingface.co/datasets/markush1/LLM-Jailbreak-Classifier.

[26] Jayavibhav, "Prompt Injection Safety Dataset", Hugging Face, https://huggingface.co/datasets/jayavibhav/prompt-injection-safety.

[27] Jayavibhav, "Prompt Injection Dataset", Hugging Face, https://huggingface.co/datasets/jayavibhav/prompt-injection.

[28] L. McInnes, J. Healy, and S. Astels, "hdbscan: Hierarchical density based clustering," *Journal of Open Source Software*, vol. 2, no. 11, p. 205, 2017.